



Quanzhou University of Information Engineering

ABCDEFGG

Lizhong Hu, Chengxiao Xu, Yunchu He

2026-05-28

- 1 Contest
- 2 Mathematics
- 3 Data structures
- 4 Numerical
- 5 Number theory
- 6 Combinatorial

- 7 Graph
- 8 Geometry

- 9 Strings
- 10 Various

Contest (1)

template.cpp53 lines

```
#include <bits/stdc++.h>
#define PV(var) cout << #var << ":" << var << " ";
#define all(arr) arr.begin(), arr.end()
#define rall(arr) arr.rbegin(), arr.rend()
#define endl "\n"
#define int long long
#define rep(i, start, end) for (int i = start; i < (end); ++i)
#define FOR(i, start, end) for (int i = start; i <= (end); ++i)
#define sz(x) (int)(x).size()
#define uni(v) v.resize(unique(all(v)) - v.begin())
using namespace std;

typedef pair<int, int> pii;
typedef vector<int> vi;
typedef long long ll;
const int inf = 4e18;
const double eps = 1e-9;

mt19937_64 gen(random_device{}());
int randint(int l, int r)
{
    return uniform_int_distribution<int>(l, r)(gen);
}

template <typename T>
istream &operator>>(istream &is, vector<T> &v)
{
    for (auto &x : v)
        is >> x;
    return is;
}

template <typename T>
ostream &operator<<(ostream &os, const vector<T> &v)
{
    if (!v.empty())
    {
        os << v[0];
```

1rep(i, 1, sz(v))

1{

1os << " " << v[i];

2}

2return os;

2}

4signed main()

4{

4cin.tie(0)->sync_with_stdio(false);

4cin.exceptions(cin.failbit);

4cout << fixed << setprecision(11);

5return 0;

5}

6hash.sh1 lines

10cpp -dD -P -fpreprocessed | tr -d '[:space:]' | md5sum | cut -c-6

11troubleshoot.txt52 lines

12Pre-submit:

Write a few simple test cases if sample is not enough.

Are time limits close? If so, generate max cases.

Is the memory usage fine?

Could anything overflow?

Make sure to submit the right file.

Wrong answer:

Print your solution! Print debug output, as well.

Are you clearing all data structures between test cases?

Can your algorithm handle the whole range of input?

Read the full problem statement again.

Do you handle all corner cases correctly?

Have you understood the problem correctly?

Any uninitialized variables?

Any overflows?

Confusing N and M, i and j, etc.?

Are you sure your algorithm works?

What special cases have you not thought of?

Are you sure the STL functions you use work as you think?

Add some assertions, maybe resubmit.

Create some testcases to run your algorithm on.

Go through the algorithm for a simple case.

Go through this list again.

Explain your algorithm to a teammate.

Ask the teammate to look at your code.

Go for a small walk, e.g. to the toilet.

Is your output format correct? (including whitespace)

Rewrite your solution from the start or let a teammate do it.

Runtime error:

Have you tested all corner cases locally?

Any uninitialized variables?

Are you reading or writing outside the range of any vector?

Any assertions that might fail?

Any possible division by 0? (mod 0 for example)

Any possible infinite recursion?

Invalidated pointers or iterators?

Are you using too much memory?

Debug with resubmits (e.g. remapped signals, see Various).

Time limit exceeded:

Do you have any possible infinite loops?

What is the complexity of your algorithm?

Are you copying a lot of unnecessary data? (References)

How big is the input and output? (consider scanf)

Avoid vector, map. (use arrays/unordered_map)

What do your teammates think about your algorithm?

Memory limit exceeded:
What is the max amount of memory your algorithm should need?
Are you clearing all data structures between test cases?

Mathematics (2)

2.1 Equations

$$ax^2 + bx + c = 0 \Rightarrow x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

The extremum is given by $x = -b/2a$.

$$\begin{aligned} ax + by &= e & x &= \frac{ed - bf}{ad - bc} \\ cx + dy &= f & y &= \frac{af - ec}{ad - bc} \end{aligned} \Rightarrow$$

2.2 Trigonometry

$$\sin(v + w) = \sin v \cos w + \cos v \sin w$$

$$\cos(v + w) = \cos v \cos w - \sin v \sin w$$

$$\tan(v + w) = \frac{\tan v + \tan w}{1 - \tan v \tan w}$$

$$\sin v + \sin w = 2 \sin \frac{v + w}{2} \cos \frac{v - w}{2}$$

$$\cos v + \cos w = 2 \cos \frac{v + w}{2} \cos \frac{v - w}{2}$$

$$a \cos x + b \sin x = r \cos(x - \phi)$$

$$a \sin x + b \cos x = r \sin(x + \phi)$$

where $r = \sqrt{a^2 + b^2}, \phi = \operatorname{atan2}(b, a)$.

2.3 Geometry

2.3.1 Triangles

Side lengths: a, b, c

Semiperimeter: $p = \frac{a + b + c}{2}$

Area (Heron): $A = \sqrt{p(p - a)(p - b)(p - c)}$

Circumradius: $R = \frac{abc}{4A}$

Inradius: $r = \frac{A}{p}$

Length of median: $m_a = \frac{1}{2}\sqrt{2b^2 + 2c^2 - a^2}$

Length of bisector: $s_a = \sqrt{bc \left[1 - \left(\frac{a}{b + c} \right)^2 \right]}$

Law of sines: $\frac{\sin \alpha}{a} = \frac{\sin \beta}{b} = \frac{\sin \gamma}{c} = \frac{1}{2R}$

Law of cosines: $a^2 = b^2 + c^2 - 2bc \cos \alpha$

2.3.2 Quadrilaterals

With side lengths a, b, c, d , diagonals e, f , diagonals angle θ , area A and magic flux $F = b^2 + d^2 - a^2 - c^2$:

$$4A = 2ef \cdot \sin \theta = F \tan \theta = \sqrt{4e^2 f^2 - F^2}$$

For cyclic quadrilaterals the sum of opposite angles is 180° , $ef = ac + bd$, and $A = \sqrt{(p-a)(p-b)(p-c)(p-d)}$.

2.4 Sums

$$c^a + c^{a+1} + \cdots + c^b = \frac{c^{b+1} - c^a}{c - 1}, c \neq 1$$

$$1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2}$$

$$1^2 + 2^2 + 3^2 + \cdots + n^2 = \frac{n(2n+1)(n+1)}{6}$$

$$1^3 + 2^3 + 3^3 + \cdots + n^3 = \frac{n^2(n+1)^2}{4}$$

$$1^4 + 2^4 + 3^4 + \cdots + n^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$$

2.5 Probability theory

Expectation is linear:

$$\mathbb{E}(aX + bY) = a\mathbb{E}(X) + b\mathbb{E}(Y)$$

For independent X and Y ,

$$V(aX + bY) = a^2V(X) + b^2V(Y).$$

Data structures (3)

OrderStatisticTree.h

Description: A set (not multiset!) with support for finding the n'th element, and finding the index of an element. To get a map, change null_type. **Time:** $\mathcal{O}(\log N)$

```
#include <bits/extc++.h>
using namespace __gnu_pbds;
```

```
template <class T>
using Tree = tree<T, null_type, less<T>, rb_tree_tag,
    tree_order_statistics_node_update>;
```

```
void example()
{
    Tree<int> t, t2;
    t.insert(8);
    auto it = t.insert(10).first;
    assert(it == t.lower_bound(9));
    assert(t.order_of_key(10) == 1);
    assert(t.order_of_key(11) == 2);
    assert(*t.find_by_order(0) == 8);
    t.join(t2); // assuming T < T2 or T > T2, merge t2 into t
}
```

HashMap.h

Description: Hash map with mostly the same API as unordered_map, but ~3x faster. Uses 1.5x memory. Initial capacity must be a power of 2 (if provided).

```
d77092, 8 lines
#include <bits/extc++.h>
// To use most bits rather than just the lowest ones:
struct chash
{ // large odd number for C
    const uint64_t C = 11(4e18 * acos(0)) | 71;
    ll operator()(ll x) const { return __builtin_bswap64(x * C);
    }
};
__gnu_pbds::gp_hash_table<ll, int, chash> h({}, {}, {}, {}, {1
    << 16});
```

SegmentTree.h

Description: Zero-indexed max-tree. Bounds are inclusive to the left and exclusive to the right. Can be changed by modifying T, f and unit.

Time: $\mathcal{O}(\log N)$

```
0f4bdb, 26 lines
struct Tree
{
    typedef int T;
    static constexpr T unit = INT_MIN;
    T f(T a, T b) { return max(a, b); } // (any associative fn)
    vector<T> s;
    int n;
    Tree(int n = 0, T def = unit) : s(2 * n, def), n(n) {}
    void update(int pos, T val)
    {
        for (s[pos += n] = val; pos /= 2;)
            s[pos] = f(s[pos * 2], s[pos * 2 + 1]);
    }
    T query(int b, int e)
    { // query [b, e)
        T ra = unit, rb = unit;
        for (b += n, e += n; b < e; b /= 2, e /= 2)
        {
            if (b % 2)
                ra = f(ra, s[b++]);
            if (e % 2)
                rb = f(s[--e], rb);
        }
        return f(ra, rb);
    }
};
```

LazySegmentTree-ACL.h

Description: Segment tree with ability to add or set values of large intervals, and compute max of intervals. Can be changed to other things.

Usage: listed

Time: $\mathcal{O}(\log N)$.

```
// 1. Range Add + Range Max
using S = ll;
using F = ll;
```

```
S op(S a, S b) { return max(a, b); }
S e() { return -4e18; }
F id() { return 0; }
```

```
S mapping(F f, S x) { return x + f; }
F composition(F f, F g) { return f + g; }
```

```
// 2. Range Add + Range Sum
```

```
struct S
{
    ll sum;
    int len;
```

```
};
using F = ll;
```

```
S op(S a, S b) { return {a.sum + b.sum, a.len + b.len}; }
S e() { return {0, 0}; }
F id() { return 0; }
```

```
S mapping(F f, S x) { return {x.sum + f * x.len, x.len}; }
F composition(F f, F g) { return f + g; }
```

```
// 3. Range Set + Range Max
```

```
using S = ll;
using F = ll;
```

```
S op(S a, S b) { return max(a, b); }
S e() { return -4e18; }
F id() { return 4e18; }
```

```
S mapping(F f, S x)
{
    return (f == id()) ? x : f;
}
F composition(F f, F g)
{
    return (f == id()) ? g : f;
}
```

```
// 4. Range Set + Range Sum
```

```
struct S
{
    ll sum;
    int len;
};
using F = ll;
```

```
S op(S a, S b) { return {a.sum + b.sum, a.len + b.len}; }
S e() { return {0, 0}; }
F id() { return 4e18; }
```

```
S mapping(F f, S x)
{
    return (f == id()) ? x : S{f * x.len, x.len};
}
F composition(F f, F g)
{
    return (f == id()) ? g : f;
}
```

```
// 5. Range Mult + Range Add + Range Sum
```

const long long MOD = 1e9 + 7;

```
struct S
{
    long long sum;
    int len;
};
struct F
{
    long long a, b; // Linear function: ax + b
    bool operator==(const F &o) const = default;
};
```

```
S op(S x, S y) { return {(x.sum + y.sum) % MOD, x.len + y.len};
    }
S e() { return {0, 0}; }
F id() { return {1, 0}; }
```

```
S mapping(F f, S x)
{
    return {f.a * x.sum + f.b * x.len, x.len};
}
```

```
    long long new_sum = (f.a * x.sum % MOD + f.b * x.len % MOD) % MOD;
    return {(new_sum + MOD) % MOD, x.len};
}
F composition(F f, F g)
{
    long long new_a = (f.a * g.a) % MOD;
    long long new_b = (f.a * g.b % MOD + f.b) % MOD;
    return {(new_a + MOD) % MOD, (new_b + MOD) % MOD};
}

// Range Add v: update(L, R, {1, v});
// Range Mult v: update(L, R, {v, 0});
// Range Set v: update(L, R, {0, v});

struct LazySeg
{
    int n;
    vector<S> d;
    vector<F> lz;
    LazySeg(int _n) : n(_n), d(4 * n, e()), lz(4 * n, id()) {}

    void apply(int p, F f)
    {
        d[p] = mapping(f, d[p]);
        lz[p] = composition(f, lz[p]);
    }

    void push(int p)
    {
        if (lz[p] != id())
        {
            apply(p << 1, lz[p]);
            apply(p << 1 | 1, lz[p]);
            lz[p] = id();
        }
    }

    void build(const vector<S> &a, int p, int cl, int cr)
    {
        if (cl == cr)
            return void(d[p] = a[cl]);
        int mid = (cl + cr) >> 1;
        build(a, p << 1, cl, mid);
        build(a, p << 1 | 1, mid + 1, cr);
        d[p] = op(d[p << 1], d[p << 1 | 1]);
    }

    void update(int l, int r, F f, int p, int cl, int cr)
    {
        if (l > cr || r < cl)
            return;

        if (l <= cl && cr <= r)
            return apply(p, f);
        push(p);
        int mid = (cl + cr) >> 1;
        update(l, r, f, p << 1, cl, mid);
        update(l, r, f, p << 1 | 1, mid + 1, cr);
        d[p] = op(d[p << 1], d[p << 1 | 1]);
    }

    S query(int l, int r, int p, int cl, int cr)
    {
        if (l > cr || r < cl)
            return e();
        if (l <= cl && cr <= r)
            return d[p];
    }
}
```

```
    push(p);
    int mid = (cl + cr) >> 1;
    return op(query(l, r, p << 1, cl, mid), query(l, r, p << 1 | 1, mid + 1, cr));
}

void build(const vector<S> &a) { build(a, 1, 0, n - 1); }
void update(int l, int r, F f) { update(l, r, f, 1, 0, n - 1); }
S query(int l, int r) { return query(l, r, 1, 0, n - 1); }
};

UnionFind.h
Description: Disjoint-set data structure.
Time:  $\mathcal{O}(\alpha(N))$ 
09c285, 45 lines

struct DSU
{
    vector<int> fa, p, e, f;
    int comp_cnt; // Number of connected components

    DSU(int n) : comp_cnt(n)
    {
        fa.resize(n + 1);
        iota(fa.begin(), fa.end(), 0);
        p.assign(n + 1, 1); // Vertices count
        e.assign(n + 1, 0); // Edges count
        f.assign(n + 1, 0); // Cycle flag
    }

    int find(int x)
    { // Path compression
        return fa[x] == x ? x : fa[x] = find(fa[x]);
    }

    bool merge(int u, int v)
    {
        int ru = find(u), rv = find(v);
        if (ru == rv)
        {
            f[ru] = 1; // Mark cycle
            e[ru]++; // Add edge
            return false;
        }
        // Union by size
        if (p[ru] < p[rv])
            swap(ru, rv);
        fa[rv] = ru;
        p[ru] += p[rv]; // Add vertices
        e[ru] += e[rv] + 1; // Add edges + 1 new edge
        f[ru] |= f[rv]; // Merge cycle flags
        comp_cnt--;
        return true;
    }

    bool same(int x, int y) { return find(x) == find(y); }
    bool has_cycle(int x) { return f[find(x)]; }
    int size(int x) { return p[find(x)]; }
    int edges(int x) { return e[find(x)]; }
    int count_sets() { return comp_cnt; }
};

SubMatrix.h
Description: Calculate submatrix sums quickly, given upper-left and lower-right corners (half-open).
Usage: SubMatrix<int> m(matrix);
m.sum(0, 0, 2, 2); // top left 4 elements
Time:  $\mathcal{O}(N^2 + Q)$ 
c59ada, 16 lines
```

```
template <class T>
struct SubMatrix
{
    vector<vector<T>> p;
    SubMatrix(vector<vector<T>> &v)
    {
        int R = sz(v), C = sz(v[0]);
        p.assign(R + 1, vector<T>(C + 1));
        rep(r, 0, R) rep(c, 0, C)
            p[r + 1][c + 1] = v[r][c] + p[r][c + 1] + p[r + 1][c] - p[r][c];
    }
    T sum(int u, int l, int d, int r)
    {
        return p[d][r] - p[d][l] - p[u][r] + p[u][l];
    }
};

Matrix.h
Description: Basic operations on square matrices.
Usage: Matrix<int, 3> A;
A.d = {{{{1,2,3}}, {{4,5,6}}, {{7,8,9}}}};
array<int, 3> vec = {1,2,3};
vec = (A^N) * vec;
4da5a2, 33 lines

template <class T, int N>
struct Matrix
{
    typedef Matrix M;
    array<array<T, N>, N> d{};
    M operator*(const M &m) const
    {
        M a;
        rep(i, 0, N) rep(j, 0, N)
            rep(k, 0, N) a.d[i][k] += d[i][j] * m.d[j][k];
        return a;
    }
    array<T, N> operator*(const array<T, N> &vec) const
    {
        array<T, N> ret{};
        rep(i, 0, N) rep(j, 0, N) ret[i] += d[i][j] * vec[j];
        return ret;
    }
    M operator^(ll p) const
    {
        assert(p >= 0);
        M a, b(*this);
        rep(i, 0, N) a.d[i][i] = 1;
        while (p)
        {
            if (p & 1)
                a = a * b;
            b = b * b;
            p >>= 1;
        }
        return a;
    }
};

FenwickTree.h
Description: Computes partial sums a[0] + a[1] + ... + a[pos - 1], and updates single elements a[i], taking the difference between the old and new value.
Time: Both operations are  $\mathcal{O}(\log N)$ .
e62fac, 30 lines

struct FT
{
    vector<ll> s;
    FT(int n) : s(n) {}
}
```

```
void update(int pos, ll dif)
{ // a[pos] += dif
  for (; pos < sz(s); pos |= pos + 1)
    s[pos] += dif;
}
ll query(int pos)
{ // sum of values in [0, pos]
  ll res = 0;
  for (; pos > 0; pos &= pos - 1)
    res += s[pos - 1];
  return res;
}
int lower_bound(ll sum)
{ // min pos st sum of [0, pos] >= sum
  // Returns n if no sum is >= sum, or -1 if empty sum is.
  if (sum <= 0)
    return -1;
  int pos = 0;
  for (int pw = 1 << 25; pw; pw >= 1)
  {
    if (pos + pw <= sz(s) && s[pos + pw - 1] < sum)
      pos += pw, sum -= s[pos - 1];
  }
  return pos;
};
```

RMQ.h
Description: Range Minimum Queries on an array. Returns min(V[a], V[a + 1], ... V[b - 1]) in constant time.
Usage: RMQ rmq(values);
 rmq.query(inclusive, exclusive);
Time: $\mathcal{O}(|V|\log|V| + Q)$

510c32, 20 lines

```
template <class T>
struct RMQ
{
  vector<vector<T>> jmp;
  RMQ(const vector<T> &V) : jmp(1, V)
  {
    for (int pw = 1, k = 1; pw * 2 <= sz(V); pw *= 2, ++k)
    {
      jmp.emplace_back(sz(V) - pw * 2 + 1);
      rep(j, 0, sz(jmp[k]))
        jmp[k][j] = min(jmp[k - 1][j], jmp[k - 1][j + pw]);
    }
  }
  T query(int a, int b)
  {
    assert(a < b); // or return inf if a == b
    int dep = 31 - __builtin_clz(b - a);
    return min(jmp[dep][a], jmp[dep][b - (1 << dep)]);
  }
};
```

Numerical (4)

4.1 Ternary Search

TernarySearch.h
Description: Ternary search for finding the extremum of a unimodal function f. Supports both discrete (integer) and continuous (double) domains. To minimize, reverse the comparison operators.

d8f024, 45 lines

```
// 1. Integer-domain Ternary Search (Finds Maximum)
// To reverse non-strict inequalities, change < in (A) to <=,
// and reverse loop (B).
// To minimize, change < to > in (A) and (B).
```

```
template <class F>
int ternSearch(int a, int b, F f)
{
  assert(a <= b);
  while (b - a >= 5)
  {
    int mid = (a + b) / 2;
    if (f(mid) < f(mid + 1))
      a = mid; // (A)
    else
      b = mid + 1;
  }
  for (int i = a + 1; i <= b; ++i)
  { // (B)
    if (f(a) < f(i))
      a = i;
  }
  return a;
}

// 2. Real-domain Ternary Search (Finds Maximum)
// To minimize, change < in (C) to >
template <class F>
double ternary_search_double(double l, double r, F f)
{
  // 100 iterations guarantee precision and prevent infinite
  // loops (eps issues)
  for (int i = 0; i < 100; ++i)
  {
    double m1 = l + (r - l) / 3.0;
    double m2 = r - (r - l) / 3.0;

    if (f(m1) < f(m2))
    {
      // (C)
      l = m1; // Max is to the right of m1
    }
    else
    {
      r = m2; // Max is to the left of m2
    }
  }
  return l;
}
```

Number theory (5)

5.1 Modular arithmetic

Modint.h

01aba0, 64 lines

```
template <int MOD>
struct Mint
{
  int v;
  Mint(long long x = 0)
  {
    v = x % MOD;
    if (v < 0)
      v += MOD;
  }
  Mint &operator+=(const Mint &o)
  {
    v += o.v;
    if (v >= MOD)
      v -= MOD;
    return *this;
  }
  Mint &operator-=(const Mint &o)
```

```
{
  v -= o.v;
  if (v < 0)
    v += MOD;
  return *this;
}
Mint &operator*=(const Mint &o)
{
  v = 1LL * v * o.v % MOD;
  return *this;
}

Mint pow(long long k) const
{
  Mint res = 1, base = *this;
  while (k)
  {
    if (k & 1)
      res *= base;
    base *= base;
    k >= 1;
  }
  return res;
}

Mint inv() const { return pow(MOD - 2); }
Mint &operator/=(const Mint &o) { return *this *= o.inv(); }

friend Mint operator+(Mint a, const Mint &b) { return a += b; }
friend Mint operator-(Mint a, const Mint &b) { return a -= b; }
friend Mint operator*(Mint a, const Mint &b) { return a *= b; }
friend Mint operator/(Mint a, const Mint &b) { return a /= b; }

friend bool operator==(const Mint &a, const Mint &b) { return a.v == b.v; }
friend bool operator!=(const Mint &a, const Mint &b) { return a.v != b.v; }

friend istream &operator>>(istream &is, Mint &x)
{
  long long v;
  is >> v;
  x = Mint(v);
  return is;
}

friend ostream &operator<<(ostream &os, const Mint &x) { return os << x.v; }
};
```

using mint = Mint<998244353>;

5.2 Primality & Divisibility

FastEratosthenes.h
Description: Prime sieve for generating all primes smaller than LIM.
Time: LIM=1e9 ≈ 1.5s

6b2912, 27 lines

```
const int LIM = 1e6;
bitset<LIM> isPrime;
vi eratosthenes()
{
  const int S = (int)round(sqrt(LIM)), R = LIM / 2;
  vi pr = {2}, sieve(S + 1);
  pr.reserve(int(LIM / log(LIM) * 1.1));
  vector<pii> cp;
  for (int i = 3; i <= S; i += 2)
```

```
    if (!sieve[i])
    {
        cp.push_back({i, i * i / 2});
        for (int j = i * i; j <= S; j += 2 * i)
            sieve[j] = 1;
    }
    for (int L = 1; L <= R; L += S)
    {
        array<bool, S> block{};
        for (auto &[p, idx] : cp)
            for (int i = idx; i < S + L; idx = (i += p))
                block[i - L] = 1;
        rep(i, 0, min(S, R - L)) if (!block[i]) pr.push_back((L + i) * 2 + 1);
    }
    for (int i : pr)
        isPrime[i] = 1;
    return pr;
}
```

euclid.h
Description: Finds two integers x and y , such that $ax + by = \gcd(a, b)$. If you just need gcd, use the built in `_gcd` instead. If a and b are coprime, then x is the inverse of $a \pmod b$.

```
33ba8f, 7 lines
11 euclid(11 a, 11 b, 11 &x, 11 &y)
{
    if (!b)
        return x = 1, y = 0, a;
    11 d = euclid(b, a % b, y, x);
    return y -= a / b * x, d;
}
```

CRT.h
Description: Chinese Remainder Theorem.
`crt(a, m, b, n)` computes x such that $x \equiv a \pmod m, x \equiv b \pmod n$. If $|a| < m$ and $|b| < n$, x will obey $0 \leq x < \text{lcm}(m, n)$. Assumes $mn < 2^{62}$.
Time: $\log(n)$

"euclid.h" 04d93a, 9 lines

```
11 crt(11 a, 11 m, 11 b, 11 n)
{
    if (n > m)
        swap(a, b), swap(m, n);
    11 x, y, g = euclid(m, n, x, y);
    assert((a - b) % g == 0); // else no solution
    x = (b - a) % n * x % n / g * m + a;
    return x < 0 ? x + m * n / g : x;
}
```

phiFunction.h
Description: Euler's ϕ function is defined as $\phi(n) := \#$ of positive integers $\leq n$ that are coprime with n . $\phi(1) = 1, p$ prime $\Rightarrow \phi(p^k) = (p - 1)p^{k-1}, m, n$ coprime $\Rightarrow \phi(mn) = \phi(m)\phi(n)$. If $n = p_1^{k_1}p_2^{k_2}...p_r^{k_r}$ then $\phi(n) = (p_1 - 1)p_1^{k_1-1}...(p_r - 1)p_r^{k_r-1}$. $\phi(n) = n \cdot \prod_{p|n} (1 - 1/p)$.
 $\sum_{d|n} \phi(d) = n, \sum_{1 \leq k \leq n, \gcd(k, n) = 1} k = n\phi(n)/2, n > 1$
Euler's thm: a, n coprime $\Rightarrow a^{\phi(n)} \equiv 1 \pmod n$.
Fermat's little thm: p prime $\Rightarrow a^{p-1} \equiv 1 \pmod p \ \forall a$.

```
cf7d6d, 11 lines
const int LIM = 5000000;
int phi[LIM];

void calculatePhi()
{
    rep(i, 0, LIM) phi[i] = i & 1 ? i : i / 2;
    for (int i = 3; i < LIM; i += 2)
        if (phi[i] == i)
            for (int j = i; j < LIM; j += i)
```

euclid CRT phiFunction Combinatorics

```
phi[j] -= phi[j] / i;
}
```

5.2.1 Bézout's identity
For $a \neq 0, b \neq 0$, then $d = \gcd(a, b)$ is the smallest positive integer for which there are integer solutions to

$$ax + by = d$$

If (x, y) is one solution, then all solutions are given by

$$\left(x + \frac{kb}{\gcd(a,b)}, y - \frac{ka}{\gcd(a,b)}\right), \quad k \in \mathbb{Z}$$

5.3 Pythagorean Triples
The Pythagorean triples are uniquely generated by

$$a = k \cdot (m^2 - n^2), \quad b = k \cdot (2mn), \quad c = k \cdot (m^2 + n^2),$$

with $m > n > 0, k > 0, m \perp n$, and either m or n even.

5.4 Primes & Hashing
 $p = 962592769$ is such that $2^{21} \mid p - 1$, which may be useful. For hashing use 970592641 (31-bit number), 31443539979727 (45-bit), 3006703054056749 (52-bit). There are 78498 primes less than 1 000 000.

Primitive roots exist modulo any prime power p^a , except for $p = 2, a > 2$, and there are $\phi(\phi(p^a))$ many. For $p = 2, a > 2$, the group $\mathbb{Z}_{2^a}^\times$ is instead isomorphic to $\mathbb{Z}_2 \times \mathbb{Z}_{2^{a-2}}$.

5.5 Estimates (Crucial for Complexity)
 $\sum_{d|n} d = O(n \log \log n)$.

The number of divisors of n is at most around 100 for $n < 5e4$, 500 for $n < 1e7$, 2000 for $n < 1e10$, 200 000 for $n < 1e19$.

Combinatorial (6)

6.1 Combinatorics (C(n, k) & P(n, k))
Combinatorics.h
Description: Precomputes factorials and inverse factorials in $O(N)$. Provides $O(1)$ queries for Combinations C(n, k) and Permutations P(n, k). Requires the 'Mint' struct.

```
d1c9b5, 35 lines
struct Comb
{
    vector<mint> f, inv_f;

    Comb(int n) : f(n + 1), inv_f(n + 1)
    {
        f[0] = 1;
        for (int i = 1; i <= n; i++)
        {
            f[i] = f[i - 1] * i;
        }
        inv_f[n] = f[n].inv(); // O(log MOD)
        // O(N) backwards computation: 1/i! = 1/(i+1)! * (i+1)
    }
}
```

```
for (int i = n - 1; i >= 0; i--)
{
    inv_f[i] = inv_f[i + 1] * (i + 1);
}

// Combination C(n, k)
mint C(int n, int k)
{
    if (k < 0 || k > n)
        return 0;
    return f[n] * inv_f[k] * inv_f[n - k];
}

// Permutation P(n, k)
mint P(int n, int k)
{
    if (k < 0 || k > n)
        return 0;
    return f[n] * inv_f[n - k];
}
};
```

6.2 Derangements
Permutations of a set such that none of the elements appear in their original position.

$$D(n) = (n - 1)(D(n - 1) + D(n - 2)) = nD(n - 1) + (-1)^n = \left\lfloor \frac{n!}{e} \right\rfloor$$

6.3 Lucas' Theorem
Let n, m be non-negative integers and p a prime. Write $n = n_kp^k + ... + n_1p + n_0$ and $m = m_kp^k + ... + m_1p + m_0$. Then

$$\binom{n}{m} \equiv \prod_{i=0}^k \binom{n_i}{m_i} \pmod p$$

6.4 Stirling numbers of the second kind
Partitions of n distinct elements into exactly k non-empty unlabelled groups.

$$S(n, k) = S(n - 1, k - 1) + kS(n - 1, k)$$

$$S(n, 1) = S(n, n) = 1$$

$$S(n, k) = \frac{1}{k!} \sum_{j=0}^k (-1)^{k-j} \binom{k}{j} j^n$$

6.5 Bell numbers
Total number of partitions of n distinct elements. $B(n) = 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, \dots$. For p prime,

$$B(p^m + n) \equiv mB(n) + B(n + 1) \pmod p$$

6.6 Labeled unrooted trees (Cayley's Formula)
on n vertices: n^{n-2}
on k existing trees of size n_i : $n_1n_2 \cdots n_k n^{k-2}$
with degrees d_i : $(n - 2)! / ((d_1 - 1)! \cdots (d_n - 1)!)$

6.7 Catalan numbers

$$C_n = \frac{1}{n+1} \binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1} = \frac{(2n)!}{(n+1)!n!}$$
$$C_0 = 1, \ C_{n+1} = \frac{2(2n+1)}{n+2} C_n, \ C_{n+1} = \sum C_i C_{n-i}$$
$$C_n = 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, \dots$$

- sub-diagonal monotone paths in an $n \times n$ grid.
- strings with n pairs of parenthesis, correctly nested.
- binary trees with with $n + 1$ leaves (0 or 2 children).
- ordered trees with $n + 1$ vertices.
- ways a convex polygon with $n + 2$ sides can be cut into triangles.
- permutations of $[n]$ with no 3-term increasing subseq.

Graph (7)

7.1 Fundamentals

Graph.h
Description: Standard Object-Oriented graph representation using adjacency lists. Prevents global variables and facilitates automatic memory cleanup in multi-test scenarios.

```
struct Graph
{
    int n, m;
    vector<vector<pii>> adj;
    vector<tuple<int, int, int>> edges;
    Graph(int n, int m = 0) : n(n), m(m), adj(n + 1) {}
    void add_edge(int u, int v, int w)
    {
        adj[u].push_back({v, w});
        edges.emplace_back(u, v, w);
    }
};
```

Dijkstra.h
Description: Dijkstra’s algorithm for single-source shortest paths on non-negative weighted graphs.
Time: $\mathcal{O}(M \log N)$

```
"Graph.h"                                     d5cb62, 23 lines

vector<int> dijkstra(int s, const Graph &g)
{
    vector<int> d(g.n + 1, inf);
    d[s] = 0;
    priority_queue<pii, vector<pii>, greater<pii>> q;
    q.push({0, s});
    while (!q.empty())
    {
        auto [dist, u] = q.top();
        q.pop();
        if (dist > d[u])
            continue;
        for (auto [v, w] : g.adj[u])
        {
            if (d[u] + w < d[v])
            {
                d[v] = d[u] + w;
                q.push({d[v], v});
            }
        }
    }
}
```

SPFA.h
Description: Shortest Path Faster Algorithm (SPFA) for graphs with negative weights. Detects negative cycles.
Time: $\mathcal{O}(NM)$ worst case

```
"Graph.h"                                     4e7e43, 32 lines

pair<bool, vector<int>> spfa(int s, const Graph &g)
{
    vector<int> dist(g.n + 1, inf), cnt(g.n + 1, 0);
    vector<bool> in_queue(g.n + 1, false);
    queue<int> q;
    dist[s] = 0;
    q.push(s);
    in_queue[s] = true;

    while (!q.empty())
    {
        int u = q.front();
        q.pop();
        in_queue[u] = false;
        for (auto [v, w] : g.adj[u])
        {
            if (dist[u] + w < dist[v])
            {
                dist[v] = dist[u] + w;
                cnt[v] = cnt[u] + 1;
                if (cnt[v] >= g.n)
                    return {false, {}}; // Negative cycle detected
                if (!in_queue[v])
                {
                    q.push(v);
                    in_queue[v] = true;
                }
            }
        }
    }
    return {true, dist};
}
```

Kruskal.h
Description: Kruskal’s algorithm for Minimum Spanning Tree (MST). Returns total weight and connectivity.
Time: $\mathcal{O}(M \log M)$

```
"Graph.h"                                     245a1e, 30 lines

pair<int, bool> kruskal(Graph &g)
{
    sort(g.edges.begin(), g.edges.end(), [](auto &a, auto &b)
        { return get<2>(a) < get<2>(b); });
    struct LocalDSU
    {
        vector<int> p;
        LocalDSU(int n) : p(n + 1) { iota(p.begin(), p.end(), 0); }
        int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); }
        bool merge(int u, int v)
        {
            int x = find(u), y = find(v);
            if (x == y)
                return false;
            p[y] = x;
            return true;
        }
    } dsu(g.n);
```

```
int total_weight = 0, comp = g.n;
for (auto [u, v, w] : g.edges)
{
    if (dsu.merge(u, v))
    {
        total_weight += w;
        --comp;
    }
}
return {total_weight, comp == 1};
}
```

Prim.h
Description: Prim’s algorithm for Minimum Spanning Tree (MST) on dense graphs.
Time: $\mathcal{O}(M \log N)$

```
"Graph.h"                                     e485a6, 29 lines

pair<int, bool> prim(int s, const Graph &g)
{
    int total_weight = 0, visited_cnt = 0;
    vector<int> dist(g.n + 1, inf);
    vector<bool> visited(g.n + 1, false);
    priority_queue<pii, vector<pii>, greater<pii>> pq;
    dist[s] = 0;
    pq.push({0, s});

    while (!pq.empty())
    {
        auto [d, u] = pq.top();
        pq.pop();
        if (visited[u])
            continue;
        visited[u] = true;
        total_weight += d;
        visited_cnt++;
        for (auto [v, w] : g.adj[u])
        {
            if (!visited[v] && w < dist[v])
            {
                dist[v] = w;
                pq.push({dist[v], v});
            }
        }
    }
    return {total_weight, visited_cnt == g.n};
}
```

TopoSort.h
Description: Topological sort. Uses vector-as-queue trick for extreme speed. Returns order, is-DAG.
Time: $\mathcal{O}(N + M)$

```
"Graph.h"                                     fa8a43, 24 lines

pair<vector<int>, bool> topo_sort(const Graph &g)
{
    vector<int> indeg(g.n + 1, 0), q;
    for (int i = 1; i <= g.n; ++i)
    {
        for (auto [v, w] : g.adj[i])
            indeg[v]++;
    }
    for (int i = 1; i <= g.n; ++i)
    {
        if (indeg[i] == 0)
            q.push_back(i);
    }
    // Vector-as-queue trick: j is the head, q.size() is the tail
    for (int j = 0; j < (int)q.size(); ++j)
```

```
{
    for (auto [v, w] : g.adj[q[j]])
    {
        if (--indeg[v] == 0)
            q.push_back(v);
    }
}
return {q, (int)q.size() == g.n};
}
```

7.2 Network flow

Dinic.h
Description: Flow algorithm with complexity $O(VE \log U)$ where $U = \max |cap|$. $O(\min(E^{1/2}, V^{2/3})E)$ if $U = 1$; $O(\sqrt{VE})$ for bipartite matching.

```
struct Dinic
{
    struct Edge
    {
        int to, rev;
        ll c, oc;
        ll flow() { return max(oc - c, 0LL); } // if you need flows
    };
    vi lvl, ptr, q;
    vector<vector<Edge>> adj;
    Dinic(int n) : lvl(n), ptr(n), q(n), adj(n) {}
    void addEdge(int a, int b, ll c, ll rcap = 0)
    {
        adj[a].push_back({b, sz(adj[b]), c, c});
        adj[b].push_back({a, sz(adj[a]) - 1, rcap, rcap});
    }
    ll dfs(int v, int t, ll f)
    {
        if (v == t || !f)
            return f;
        for (int &i = ptr[v]; i < sz(adj[v]); i++)
        {
            Edge &e = adj[v][i];
            if (lvl[e.to] == lvl[v] + 1)
                if (ll p = dfs(e.to, t, min(f, e.c)))
                {
                    e.c -= p, adj[e.to][e.rev].c += p;
                    return p;
                }
        }
        return 0;
    }
    ll calc(int s, int t)
    {
        ll flow = 0;
        q[0] = s;
        rep(L, 0, 31) do
        { // 'int L=30' maybe faster for random data
            lvl = ptr = vi(sz(q));
            int qi = 0, qe = lvl[s] = 1;
            while (qi < qe && !lvl[t])
            {
                int v = q[qi++];
                for (Edge e : adj[v])
                    if (!lvl[e.to] && e.c >> (30 - L))
                        q[qe++] = e.to, lvl[e.to] = lvl[v] + 1;
            }
            while (ll p = dfs(s, t, LLONG_MAX))
                flow += p;
        }
        while (lvl[t])
            ;
        return flow;
    }
};
```

```
}
bool leftOfMinCut(int a) { return lvl[a] != 0; }
};
```

MinCostMaxFlow.h
Description: Min-cost max-flow. If costs can be negative, call setpi before maxflow, but note that negative cost cycles are not supported. To obtain the actual flow, look at positive values only.
Time: $\mathcal{O}(FE \log(V))$ where F is max flow. $\mathcal{O}(VE)$ for setpi.

```
#include <bits/extc++.h>

const ll INF = numeric_limits<ll>::max() / 4;

struct MCMF
{
    struct edge
    {
        int from, to, rev;
        ll cap, cost, flow;
    };
    int N;
    vector<vector<edge>> ed;
    vi seen;
    vector<ll> dist, pi;
    vector<edge*> par;

    MCMF(int N) : N(N), ed(N), seen(N), dist(N), pi(N), par(N) {}

    void addEdge(int from, int to, ll cap, ll cost)
    {
        if (from == to)
            return;
        ed[from].push_back(edge{from, to, sz(ed[to]), cap, cost, 0});
        ed[to].push_back(edge{to, from, sz(ed[from]) - 1, 0, -cost, 0});
    }

    void path(int s)
    {
        fill(all(seen), 0);
        fill(all(dist), INF);
        dist[s] = 0;
        ll di;

        __gnu_pbds::priority_queue<pair<ll, int>> q;
        vector<decltype(q)::point_iterator> its(N);
        q.push({0, s});

        while (!q.empty())
        {
            s = q.top().second;
            q.pop();
            seen[s] = 1;
            di = dist[s] + pi[s];
            for (edge &e : ed[s])
                if (!seen[e.to])
                {
                    ll val = di - pi[e.to] + e.cost;
                    if (e.cap - e.flow > 0 && val < dist[e.to])
                    {
                        dist[e.to] = val;
                        par[e.to] = &e;
                        if (its[e.to] == q.end())
                            its[e.to] = q.push({-dist[e.to], e.to});
                        else
                            q.modify(its[e.to], {-dist[e.to], e.to});
                    }
                }
        }
    }
};
```

```
}

rep(i, 0, N) pi[i] = min(pi[i] + dist[i], INF);
}

pair<ll, ll> maxflow(int s, int t)
{
    ll totflow = 0, totcost = 0;
    while (path(s), seen[t])
    {
        ll fl = INF;
        for (edge *x = par[t]; x; x = par[x->from])
            fl = min(fl, x->cap - x->flow);

        totflow += fl;
        for (edge *x = par[t]; x; x = par[x->from])
        {
            x->flow += fl;
            ed[x->to][x->rev].flow -= fl;
        }
        rep(i, 0, N) for (edge &e : ed[i]) totcost += e.cost * e.flow;
        return {totflow, totcost / 2};
    }

    // If some costs can be negative, call this before maxflow:
    void setpi(int s)
    { // (otherwise, leave this out)
        fill(all(pi), INF);
        pi[s] = 0;
        int it = N, ch = 1;
        ll v;
        while (ch-- && it--)
            rep(i, 0, N) if (pi[i] != INF) for (edge &e : ed[i]) if (e.cap) if ((v = pi[i] + e.cost) < pi[e.to])
                pi[e.to] = v,
                ch = 1;
        assert(it >= 0); // negative cost cycle
    }
};
```

7.3 Matching

HopcroftKarp.h
Description: Fast bipartite matching algorithm. Graph g should be a list of neighbors of the left partition, and r should be a vector full of -1 's of the same size as the right partition. Returns the size of the matching. $r[i]$ will be the match for vertex i on the right side, or -1 if it's not matched.

```
Time:  $\mathcal{O}(E\sqrt{V})$ 
int hopcroftKarp(vector<vi> &g, vi &r)
{
    int n = sz(g), res = 0;
    vi l(n, -1), q(n), d(n);
    auto dfs = [&](auto f, int u) -> bool
    {
        int t = exchange(d[u], 0) + 1;
        for (int v : g[u])
            if (r[v] == -1 || (d[r[v]] == t && f(f, r[v])))
                return l[u] = v, r[v] = u, 1;
        return 0;
    };
    for (int t = 0, f = 0;; t = f = 0, d.assign(n, 0))
    {
        rep(i, 0, n) if (l[i] == -1) q[t++] = i, d[i] = 1;
        rep(i, 0, t) for (int v : g[q[i]])
        {
            if (r[v] == -1)
```

```
        f = 1;
    else if (!d[r[v]])
        d[r[v]] = d[q[i]] + 1, q[t++] = r[v];
    }
    if (!f)
        return res;
    rep(i, 0, n) if (l[i] == -1) res += dfs(dfs, i);
}
}
```

WeightedMatching.h

Description: Given a weighted bipartite graph, matches every node on the left with a node on the right such that no nodes are in two matchings and the sum of the edge weights is minimal. Takes cost[N][M], where cost[i][j] = cost for L[i] to be matched with R[j] and returns (min cost, match), where L[i] is matched with R[match[i]]. Negate costs for max cost. Requires $N \leq M$.
Time: $\mathcal{O}(N^2M)$

1e0fe9, 42 lines

```
pair<int, vi> hungarian(const vector<vi> &a)
{
    if (a.empty())
        return {0, {}};
    int n = sz(a) + 1, m = sz(a[0]) + 1;
    vi u(n), v(m), p(m), ans(n - 1);
    rep(i, 1, n)
    {
        p[0] = i;
        int j0 = 0; // add "dummy" worker 0
        vi dist(m, INT_MAX), pre(m, -1);
        vector<bool> done(m + 1);
        do
        { // dijkstra
            done[j0] = true;
            int i0 = p[j0], j1, delta = INT_MAX;
            rep(j, 1, m) if (!done[j])
            {
                auto cur = a[i0 - 1][j - 1] - u[i0] - v[j];
                if (cur < dist[j])
                    dist[j] = cur, pre[j] = j0;
                if (dist[j] < delta)
                    delta = dist[j], j1 = j;
            }
            rep(j, 0, m)
            {
                if (done[j])
                    u[p[j]] += delta, v[j] -= delta;
                else
                    dist[j] -= delta;
            }
            j0 = j1;
        } while (p[j0]);
        while (j0)
        { // update alternating path
            int j1 = pre[j0];
            p[j0] = p[j1], j0 = j1;
        }
        rep(j, 1, m) if (p[j]) ans[p[j] - 1] = j - 1;
        return {-v[0], ans}; // min cost
    }
}
```

MinimumVertexCover.h

Description: Finds a minimum vertex cover in a bipartite graph. The size is the same as the size of a maximum matching, and the complement is a maximum independent set.

"DFSMatching.h" da4196, 27 lines

```
vi cover(vector<vi> &g, int n, int m)
{
    vi match(m, -1);
```

```
int res = dfsMatching(g, match);
vector<bool> lfound(n, true), seen(m);
for (int it : match)
    if (it != -1)
        lfound[it] = false;
vi q, cover;
rep(i, 0, n) if (lfound[i]) q.push_back(i);
while (!q.empty())
{
    int i = q.back();
    q.pop_back();
    lfound[i] = 1;
    for (int e : g[i])
        if (!seen[e] && match[e] != -1)
        {
            seen[e] = true;
            q.push_back(match[e]);
        }
}
rep(i, 0, n) if (!lfound[i]) cover.push_back(i);
rep(i, 0, m) if (seen[i]) cover.push_back(n + i);
assert(sz(cover) == res);
return cover;
}
```

7.4 Connectivity & DFS algorithms

SCC.h

Description: Finds strongly connected components in a directed graph. If vertices u, v belong to the same component, we can reach u from v and vice versa.
Usage: scc(graph, [&](vi& v) { ... }) visits all components in reverse topological order. comp[i] holds the component index of a node (a component only has edges to components with lower index). ncomps will contain the number of components.
Time: $\mathcal{O}(E + V)$

76b5c9, 35 lines

```
vi val, comp, z, cont;
int Time, ncomps;
template <class G, class F>
int dfs(int j, G &g, F &f)
{
    int low = val[j] = ++Time, x;
    z.push_back(j);
    for (auto e : g[j])
        if (comp[e] < 0)
            low = min(low, val[e] ?: dfs(e, g, f));

    if (low == val[j])
    {
        do
        {
            x = z.back();
            z.pop_back();
            comp[x] = ncomps;
            cont.push_back(x);
        } while (x != j);
        f(cont);
        cont.clear();
        ncomps++;
    }
    return val[j] = low;
}
template <class G, class F>
void scc(G &g, F f)
{
    int n = sz(g);
    val.assign(n, 0);
    comp.assign(n, -1);
    Time = ncomps = 0;
```

```
    rep(i, 0, n) if (comp[i] < 0) dfs(i, g, f);
}
```

BCC.h

Description: Finds all vertex-biconnected components (V-BCC/BCC) in an undirected graph, and automatically identifies all bridges. Completely encapsulated in a struct to prevent multi-test clearing bugs.
Usage: BCC bcc(N);
for each edge (u, v): bcc.add_edge(u, v);
bcc.build(callback); // callback receives vector<int> of edge IDs
// bcc.bridges now contains all bridges as pair<int, int>
Time: $\mathcal{O}(V + E)$

9e633e, 68 lines

```
struct BCC
{
    int n, edge_id = 0, timestamp = 0;
    vector<vector<pii>> adj;
    vector<int> num, st;
    vector<pii> bridges; // Automatically populated during build()

    BCC(int n) : n(n), adj(n + 1) {}

    void add_edge(int u, int v)
    {
        adj[u].push_back({v, edge_id});
        adj[v].push_back({u, edge_id++});
    }

    template <class F>
    int dfs(int at, int par_edge, F &f)
    {
        int me = num[at] = ++timestamp, top = me;
        for (auto [y, e] : adj[at])
        {
            if (e != par_edge)
            {
                if (num[y])
                {
                    top = min(top, num[y]);
                    if (num[y] < me)
                        st.push_back(e);
                }
                else
                {
                    int si = st.size();
                    int up = dfs(y, e, f);
                    top = min(top, up);
                    if (up == me)
                    {
                        st.push_back(e);
                        f(vector<int>(st.begin() + si, st.end()));
                        st.resize(si);
                    }
                    else if (up < me)
                    {
                        st.push_back(e);
                    }
                    else
                    {
                        bridges.push_back({at, y}); // e is a bridge!
                    }
                }
            }
        }
        return top;
    }
}
```

```
    }

    template <class F>
    void build(F f)
    {
        num.assign(n + 1, 0);
        st.clear();
        bridges.clear();
        timestamp = 0;
        for (int i = 1; i <= n; ++i)
        {
            if (!num[i])
                dfs(i, -1, f);
        }
    }
};
```

2sat.h
Description: Calculates a valid assignment to boolean variables a, b, c,... to a 2-SAT problem, so that an expression of the type $(a||b)\&\&(!a||c)\&\&(d||b)\&\&...$ becomes true, or reports that it is unsatisfiable. Negated variables are represented by bit-inversions (~x).
Usage: TwoSat ts(number of boolean variables);
ts.either(0, ~3); // Var 0 is true or var 3 is false
ts.setValue(2); // Var 2 is true
ts.atMostOne({0,~1,2}); // <= 1 of vars 0, ~1 and 2 are true
ts.solve(); // Returns true iff it is solvable
ts.values[0..N-1] holds the assigned values to the vars
Time: $\mathcal{O}(N + E)$, where N is the number of boolean variables, and E is the number of clauses.

5f9706, 71 lines

```
struct TwoSat
{
    int N;
    vector<vi> gr;
    vi values; // 0 = false, 1 = true

    TwoSat(int n = 0) : N(n), gr(2 * n) {}

    int addVar()
    { // (optional)
        gr.emplace_back();
        gr.emplace_back();
        return N++;
    }

    void either(int f, int j)
    {
        f = max(2 * f, -1 - 2 * f);
        j = max(2 * j, -1 - 2 * j);
        gr[f].push_back(j ^ 1);
        gr[j].push_back(f ^ 1);
    }

    void setValue(int x) { either(x, x); }

    void atMostOne(const vi &li)
    { // (optional)
        if (sz(li) <= 1)
            return;
        int cur = ~li[0];
        rep(i, 2, sz(li))
        {
            int next = addVar();
            either(cur, ~li[i]);
            either(cur, next);
            either(~li[i], next);
            cur = ~next;
        }
        either(cur, ~li[1]);
    }
};
```

```
    }

    vi val, comp, z;
    int time = 0;
    int dfs(int i)
    {
        int low = val[i] = ++time, x;
        z.push_back(i);
        for (int e : gr[i])
            if (!comp[e])
                low = min(low, val[e] ?: dfs(e));
        if (low == val[i])
            do
            {
                x = z.back();
                z.pop_back();
                comp[x] = low;
                if (values[x >> 1] == -1)
                    values[x >> 1] = x & 1;
            } while (x != i);
        return val[i] = low;
    }

    bool solve()
    {
        values.assign(N, -1);
        val.assign(2 * N, 0);
        comp = val;
        rep(i, 0, 2 * N) if (!comp[i]) dfs(i);
        rep(i, 0, N) if (comp[2 * i] == comp[2 * i + 1]) return 0;
        return 1;
    }
};
```

EulerWalk.h
Description: Eulerian undirected/directed path/cycle algorithm. Input should be a vector of (dest, global edge index), where for undirected graphs, forward/backward edges have the same index. Returns a list of nodes in the Eulerian path/cycle with src at both start and end, or empty list if no cycle/path exists. To get edge indices back, add .second to s and ret.
Time: $\mathcal{O}(V + E)$

780b64, 27 lines

```
vi eulerWalk(vector<vector<pii>> &gr, int nedges, int src = 0)
{
    int n = sz(gr);
    vi D(n), its(n), eu(nedges), ret, s = {src};
    D[src]++; // to allow Euler paths, not just cycles
    while (!s.empty())
    {
        int x = s.back(), y, e, &it = its[x], end = sz(gr[x]);
        if (it == end)
        {
            ret.push_back(x);
            s.pop_back();
            continue;
        }
        tie(y, e) = gr[x][it++];
        if (!eu[e])
        {
            D[x]--, D[y]++;
            eu[e] = 1;
            s.push_back(y);
        }
    }
    for (int x : D)
        if (x < 0 || sz(ret) != nedges + 1)
            return {};
    return {ret.rbegin(), ret.rend()};
}
```

7.5 Trees
LCA.h
Description: Data structure for computing lowest common ancestors in a tree (with 0 as root). C should be an adjacency list of the tree, either directed or undirected.
Time: $\mathcal{O}(N \log N + Q)$

../data-structures/RMQ.h" 0f62fb, 21 lines

```
struct LCA {
    int T = 0;
    vi time, path, ret;
    RMQ<int> rmq;

    LCA(vector<vi>& C) : time(sz(C)), rmq((dfs(C,0,-1), ret)) {}
    void dfs(vector<vi>& C, int v, int par) {
        time[v] = T++;
        for (int y : C[v]) if (y != par) {
            path.push_back(v), ret.push_back(time[v]);
            dfs(C, y, v);
        }
    }

    int lca(int a, int b) {
        if (a == b) return a;
        tie(a, b) = minmax(time[a], time[b]);
        return path[rmq.query(a, b)];
    }
    //dist(a,b){return depth[a] + depth[b] - 2*depth[lca(a,b)];}
};
```

TreeDiameter.h
Description: Comprehensive tree analyzer. Computes tree diameter, radius, center, and centroid in $\mathcal{O}(N)$.
Time: $\mathcal{O}(N)$

9e47d5, 112 lines

```
struct Tree
{
    int n;
    vector<vector<pii>> e;
    int diam = 0, radius = inf, center = 0;
    vector<int> dl, d2, up, sl;

    int cog = 0, rem_max_sz = inf;
    vector<int> sz;

    Tree(int n) : n(n), e(n + 1) {}

    void add(int u, int v, int w = 1)
    {
        e[u].push_back({v, w});
        e[v].push_back({u, w});
    }

    void _dfs1(int u, int fa)
    {
        for (auto [v, w] : e[u])
        {
            if (v == fa)
                continue;
            _dfs1(v, u);
            int val = dl[v] + w;
            if (val > dl[u])
            {
                d2[u] = dl[u];
                dl[u] = val;
                sl[u] = v;
            }
            else if (val > d2[u])
            {

```

```

        d2[u] = val;
    }
}

void _dfs2(int u, int fa)
{
    for (auto [v, w] : e[u])
    {
        if (v == fa)
            continue;
        if (s1[u] == v)
        {
            up[v] = max(up[u], d2[u]) + w;
        }
        else
        {
            up[v] = max(up[u], d1[u]) + w;
        }
        _dfs2(v, u);
    }
}

// Computes tree diameter, radius, and center
void get_diam_info(int root = 1)
{
    d1.assign(n + 1, 0);
    d2.assign(n + 1, 0);
    up.assign(n + 1, 0);
    s1.assign(n + 1, 0);
    diam = 0;
    radius = inf;
    center = 0;

    _dfs1(root, 0);
    _dfs2(root, 0);

    for (int i = 1; i <= n; ++i)
    {
        diam = max(diam, d1[i] + up[i]);
        int current_rad = max(d1[i], up[i]);
        if (current_rad < radius)
        {
            radius = current_rad;
            center = i;
        }
    }

    void _dfs_cog(int u, int fa)
    {
        sz[u] = 1;
        int max_child_sz = 0;
        for (auto [v, w] : e[u])
        {
            if (v == fa)
                continue;
            _dfs_cog(v, u);
            sz[u] += sz[v];
            max_child_sz = max(max_child_sz, sz[v]);
        }
        int max_sz = max(max_child_sz, n - sz[u]);
        if (max_sz < rem_max_sz)
        {
            rem_max_sz = max_sz;
            cog = u;
        }
    }
}

```

```

// Finds the tree centroid (cog)
void get_cog_info(int root = 1)
{
    sz.assign(n + 1, 0);
    rem_max_sz = inf;
    cog = 0;
    _dfs_cog(root, 0);
}
};

```

7.6 Math

7.6.1 Erdős–Gallai theorem

A simple graph with node degrees $d_1 \geq \dots \geq d_n$ exists iff $d_1 + \dots + d_n$ is even and for every $k = 1 \dots n$,

$$\sum_{i=1}^k d_i \leq k(k-1) + \sum_{i=k+1}^n \min(d_i, k).$$

Geometry (8)

8.1 Geometric primitives

Point.h

Description: Class to handle points in the plane. T can be e.g. double or long long. (Avoid int.)

```

template <class T> int sgn(T x) { return (x > 0) - (x < 0); }
template <class T>
struct Point {
    typedef Point P;
    T x, y;
    explicit Point(T x=0, T y=0) : x(x), y(y) {}
    bool operator<(P p) const { return tie(x,y) < tie(p.x,p.y); }
    bool operator==(P p) const { return tie(x,y)==tie(p.x,p.y); }
    P operator+(P p) const { return P(x+p.x, y+p.y); }
    P operator-(P p) const { return P(x-p.x, y-p.y); }
    P operator*(T d) const { return P(x*d, y*d); }
    P operator/(T d) const { return P(x/d, y/d); }
    T dot(P p) const { return x*p.x + y*p.y; }
    T cross(P p) const { return x*p.y - y*p.x; }
    T cross(P a, P b) const { return (a-*this).cross(b-*this); }
    T dist2() const { return x*x + y*y; }
    double dist() const { return sqrt((double)dist2()); }
    // angle to x-axis in interval [-pi, pi]
    double angle() const { return atan2(y, x); }
    P unit() const { return *this/dist(); } // makes dist()==1
    P perp() const { return P(-y, x); } // rotates +90 degrees
    P normal() const { return perp().unit(); }
    // returns point rotated 'a' radians ccw around the origin
    P rotate(double a) const {
        return P(x*cos(a)-y*sin(a), x*sin(a)+y*cos(a)); }
    friend ostream& operator<<(ostream& os, P p) {
        return os << "(" << p.x << ", " << p.y << ")"; }
};

```

SegmentDistance.h

Description:

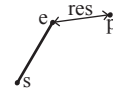
Returns the shortest distance between point p and the line segment from point s to e.

Usage: Point<double> a, b(2,2), p(1,1);
bool onSegment = segDist(a,b,p) < 1e-10;

```

"Point.h"
typedef Point<double> P;
double segDist(P& s, P& e, P& p) {

```



```

if (s==e) return (p-s).dist();
auto d = (e-s).dist2(), t = min(d,max(.0, (p-s).dot(e-s)));
return ((p-s)*d-(e-s)*t).dist()/d;
}

```

SegmentIntersection.h

Description:

If a unique intersection point between the line segments going from s1 to e1 and from s2 to e2 exists then it is returned. If no intersection point exists an empty vector is returned. If infinitely many exist a vector with 2 elements is returned, containing the endpoints of the common line segment. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or long long.

Usage: vector<P> inter = segInter(s1,e1,s2,e2);

```

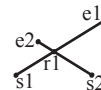
if (sz(inter)==1)
cout << "segments intersect at " << inter[0] << endl;
"Point.h", "OnSegment.h"
9d57f2, 13 lines

```

```

template<class P> vector<P> segInter(P a, P b, P c, P d) {
    auto oa = c.cross(d, a), ob = c.cross(d, b),
        oc = a.cross(b, c), od = a.cross(b, d);
    // Checks if intersection is single non-endpoint point.
    if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0)
        return {(a * ob - b * oa) / (ob - oa)};
    set<P> s;
    if (onSegment(c, d, a)) s.insert(a);
    if (onSegment(c, d, b)) s.insert(b);
    if (onSegment(a, b, c)) s.insert(c);
    if (onSegment(a, b, d)) s.insert(d);
    return {all(s)};
}

```



lineIntersection.h

Description:

If a unique intersection point of the lines going through s1,e1 and s2,e2 exists {1, point} is returned. If no intersection point exists {0, (0,0)} is returned and if infinitely many exists {-1, (0,0)} is returned. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or ll.

Usage: auto res = lineInter(s1,e1,s2,e2);

```

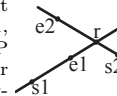
if (res.first == 1)
cout << "intersection point at " << res.second << endl;
"Point.h"
a01f81, 8 lines

```

```

template<class P>
pair<int, P> lineInter(P s1, P e1, P s2, P e2) {
    auto d = (e1 - s1).cross(e2 - s2);
    if (d == 0) // if parallel
        return {-(s1.cross(e1, s2) == 0), P(0, 0)};
    auto p = s2.cross(e1, e2), q = s2.cross(e2, s1);
    return {1, (s1 * p + e1 * q) / d};
}

```



OnSegment.h

Description: Returns true iff p lies on the line segment from s to e. Use (segDist(s,e,p)<=epsilon) instead when using Point<double>.

```

"Point.h"
c597e8, 3 lines
template<class P> bool onSegment(P s, P e, P p) {
    return p.cross(s, e) == 0 && (s - p).dot(e - p) <= 0;
}

```

8.2 Polygons

InsidePolygon.h

Description: Returns true if p lies within the polygon. If strict is true, it returns false for points on the boundary. The algorithm uses products in intermediate steps so watch out for overflow.

Usage: vector<P> v = {P{4,4}, P{1,2}, P{2,1}};

bool in = inPolygon(v, P{3, 3}, false);

Time: $\mathcal{O}(n)$

"Point.h", "OnSegment.h", "SegmentDistance.h"	2bf504, 11 lines
<pre>template<class P> bool inPolygon(vector<P> &p, P a, bool strict = true) { int cnt = 0, n = sz(p); rep(i,0,n) { P q = p[(i + 1) % n]; if (onSegment(p[i], q, a)) return !strict; //or: if (segDist(p[i], q, a) <= eps) return !strict; cnt ^= ((a.y<p[i].y) - (a.y<q.y)) * a.cross(p[i], q) > 0; } return cnt; }</pre>	

PolygonArea.h

Description: Returns twice the signed area of a polygon. Clockwise enumeration gives negative area. Watch out for overflow if using int as T!

"Point.h"	f12300, 6 lines
<pre>template<class T> T polygonArea2(vector<Point<T>>& v) { T a = v.back().cross(v[0]); rep(i,0,sz(v)-1) a += v[i].cross(v[i+1]); return a; }</pre>	

ConvexHull.h

Description: Returns a vector of the points of the convex hull in counter-clockwise order. Points on the edge of the hull between two other points are not considered part of the hull.

Time: $\mathcal{O}(n \log n)$

"Point.h"	310954, 13 lines
<pre>typedef Point<ll> P; vector<P> convexHull(vector<P> pts) { if (sz(pts) <= 1) return pts; sort(all(pts)); vector<P> h(sz(pts)+1); int s = 0, t = 0; for (int it = 2; it--; s = --t, reverse(all(pts))) for (P p : pts) { while (t >= s + 2 && h[t-2].cross(h[t-1], p) <= 0) t--; h[t++] = p; } return {h.begin(), h.begin() + t - (t == 2 && h[0] == h[1])}; }</pre>	



Strings (9)

KMP.h

Description: pi[x] computes the length of the longest prefix of s that ends at x, other than s[0...x] itself (abacaba -> 0010123). Can be used to find all occurrences of a string.

Time: $\mathcal{O}(n)$

d4375c, 19 lines
<pre>vi pi(const string &s) { vi p(sz(s)); rep(i, 1, sz(s)) {</pre>

<pre>int g = p[i - 1]; while (g && s[i] != s[g]) g = p[g - 1]; p[i] = g + (s[i] == s[g]); } return p; }</pre>

<pre>vi match(const string &s, const string &pat) { vi p = pi(pat + '\0' + s), res; rep(i, sz(p) - sz(s), sz(p)) if (p[i] == sz(pat)) res. push_back(i - 2 * sz(pat)); return res; }</pre>
--

Zfunc.h

Description: z[i] computes the length of the longest common prefix of s[i:] and s, except z[0] = 0. (abacaba -> 0010301)

Time: $\mathcal{O}(n)$

ee09e2, 14 lines
<pre>vi Z(const string &S) { vi z(sz(S)); int l = -1, r = -1; rep(i, 1, sz(S)) { z[i] = i >= r ? 0 : min(r - i, z[i - l]); while (i + z[i] < sz(S) && S[i + z[i]] == S[z[i]]) z[i]++; if (i + z[i] > r) l = i, r = i + z[i]; } return z; }</pre>

Manacher.h

Description: For each position in a string, computes p[0][i] = half length of longest even palindrome around pos i, p[1][i] = longest odd (half rounded down).

Time: $\mathcal{O}(N)$

e7ad79, 17 lines
<pre>array<vi, 2> manacher(const string &s) { int n = sz(s); array<vi, 2> p = {vi(n + 1), vi(n)}; rep(z, 0, 2) for (int i = 0, l = 0, r = 0; i < n; i++) { int t = r - i + !z; if (i < r) p[z][i] = min(t, p[z][l + t]); int L = i - p[z][i], R = i + p[z][i] - !z; while (L >= 1 && R + 1 < n && s[L - 1] == s[R + 1]) p[z][i]++, L--, R++; if (R > r) l = L, r = R; } return p; }</pre>

MinRotation.h

Description: Finds the lexicographically smallest rotation of a string.

Usage: rotate(v.begin(), v.begin()+minRotation(v), v.end());

Time: $\mathcal{O}(N)$

d07a42, 19 lines
<pre>int minRotation(string s) { int a = 0, N = sz(s); s += s;</pre>

<pre>rep(b, 0, N) rep(k, 0, N) { if (a + k == b s[a + k] < s[b + k]) { b += max(0, k - 1); break; } if (s[a + k] > s[b + k]) { a = b; break; } } return a; }</pre>

Hashing.h

Description: Extremely fast string hashing modulo $2^{64} - 1$ using 128-bit multiplication fold. Completely immune to Thue-Morse anti-hash tests. Base C is automatically randomized at runtime to prevent any targeted hacks. Range query 'hashInterval(a, b)' returns hash of half-open interval $[a, b)$ (i.e. 0-based $[a, b - 1]$).

Time: $\mathcal{O}(N)$ precomputation, $\mathcal{O}(1)$ query

3b2d78, 66 lines
<pre>typedef uint64_t ull; struct H { ull x; H(ull x = 0) : x(x) {} H operator+(H o) { return x + o.x + (x + o.x < x); } H operator-(H o) { return *this + ~o.x; } H operator*(H o) { auto m = (__uint128_t)x * o.x; return H((ull)m + (ull)(m >> 64)); } ull get() const { return x + !~x; } bool operator==(H o) const { return get() == o.get(); } bool operator<(H o) const { return get() < o.get(); } }; inline const H C = []() { mt19937_64 rng(random_device{}()); return H(rng() 1); // Ensure C is odd })(); struct HashInterval { vector<H> ha, pw; HashInterval(const string &str) : ha(str.size() + 1), pw(ha) { pw[0] = 1; for (int i = 0; i < (int)str.size(); ++i) { ha[i + 1] = ha[i] * C + str[i]; pw[i + 1] = pw[i] * C; } } H hashInterval(int a, int b) { // hash [a, b) in 0-based indexing return ha[b] - ha[a] * pw[b - a]; } };</pre>

<pre>vector<H> getHashes(const string &str, int length) { if ((int)str.size() < length) return {};</pre>

```
H h = 0, pw = 1;
for (int i = 0; i < length; ++i)
{
    h = h * C + str[i];
    pw = pw * C;
}
vector<H> ret = {h};
for (int i = length; i < (int)str.size(); ++i)
{
    ret.push_back(h = h * C + str[i] - pw * str[i - length]);
}
return ret;
}
```

```
H hashString(const string &s)
{
    H h{};
    for (char c : s)
        h = h * C + c;
    return h;
}
```

Trie.h
Description: Prefix Trie for lowercase English strings. Keeps track of the count of ending strings.
Time: $\mathcal{O}(L)$ per operation where L is string length. Space: $\mathcal{O}(L * 26)$.

```
struct Trie
{
    vector<array<int, 26>> ch;
    vector<int> cnt;

    Trie() : ch(1), cnt(1, 0)
    {
        ch[0].fill(0);
    }

    void insert(const string &s)
    {
        int u = 0;
        for (char c : s)
        {
            int v = c - 'a';
            if (!ch[u][v])
            {
                ch[u][v] = ch.size();
                ch.emplace_back();
                ch.back().fill(0);
                cnt.push_back(0);
            }
            u = ch[u][v];
        }
        cnt[u]++;

        bool search(const string &s)
        {
            int u = 0;
            for (char c : s)
            {
                int v = c - 'a';
                if (!ch[u][v])
                    return false;
                u = ch[u][v];
            }
            return cnt[u] > 0;
        }
    };
};
```

Various (10)

10.1 Intervals

IntervalContainer.h
Description: Add and remove intervals from a set of disjoint intervals. Will merge the added interval with any overlapping intervals in the set when adding. Intervals are [inclusive, exclusive).
Time: $\mathcal{O}(\log N)$

```
set<pii>::iterator addInterval(set<pii> &is, int L, int R)
{
    if (L == R)
        return is.end();
    auto it = is.lower_bound({L, R}), before = it;
    while (it != is.end() && it->first <= R)
    {
        R = max(R, it->second);
        before = it = is.erase(it);
    }
    if (it != is.begin() && (--it)->second >= L)
    {
        L = min(L, it->first);
        R = max(R, it->second);
        is.erase(it);
    }
    return is.insert(before, {L, R});
}
```

```
void removeInterval(set<pii> &is, int L, int R)
{
    if (L == R)
        return;
    auto it = addInterval(is, L, R);
    auto r2 = it->second;
    if (it->first == L)
        is.erase(it);
    else
        (int &)it->second = L;
    if (R != r2)
        is.emplace(R, r2);
}
```

IntervalCover.h
Description: Compute indices of smallest set of intervals covering another interval. Intervals should be [inclusive, exclusive). To support [inclusive, inclusive], change (A) to add || R.empty(). Returns empty set on failure (or if G is empty).
Time: $\mathcal{O}(N \log N)$

```
template<class T>
vi cover(pair<T, T> G, vector<pair<T, T>> I) {
    vi S(sz(I)), R;
    iota(all(S), 0);
    sort(all(S), [&](int a, int b) { return I[a] < I[b]; });
    T cur = G.first;
    int at = 0;
    while (cur < G.second) { // (A)
        pair<T, int> mx = make_pair(cur, -1);
        while (at < sz(I) && I[S[at]].first <= cur) {
            mx = max(mx, make_pair(I[S[at]].second, S[at]));
            at++;
        }
        if (mx.second == -1) return {};
        cur = mx.first;
        R.push_back(mx.second);
    }
    return R;
}
```

10.2 Misc. algorithms

LIS.h
Description: Compute indices for the longest increasing subsequence.
Time: $\mathcal{O}(N \log N)$

```
template <class I>
vi lis(const vector<I> &S)
{
    if (S.empty())
        return {};
    vi prev(sz(S));
    typedef pair<I, int> p;
    vector<p> res;
    rep(i, 0, sz(S))
    {
        // change 0 -> i for longest non-decreasing subsequence
        auto it = lower_bound(all(res), p{S[i], 0});
        if (it == res.end())
            res.emplace_back(), it = res.end() - 1;
        *it = {S[i], i};
        prev[i] = it == res.begin() ? 0 : (it - 1)->second;
    }
    int L = sz(res), cur = res.back().second;
    vi ans(L);
    while (L--)
        ans[L] = cur, cur = prev[cur];
    return ans;
}
```

10.3 Debugging tricks

- signal(SIGSEGV, [](int) { _Exit(0); }); converts segfaults into Wrong Answers.
- feenableexcept(29); kills the program on NaNs (1), 0-divs (4), infinities (8) and denormals (16) in local debug.

10.4 Optimization tricks

__builtin_ia32_ldmxcsr(40896); disables denormals (makes floats 20x faster near minimum value).

10.4.1 Bit hacks

- x & -x is the least bit in x.
- for (int x = m; x;) { --x &= m; ... } loops over all subset masks of m (except m itself).
- rep(b,0,K) rep(i,0,(1 << K)) if (i & 1 << b) D[i] SOS DP (Sum Over Subsets).

10.4.2 Pragmas

- #pragma GCC optimize ("Ofast") will make GCC auto-vectorize loops and optimize floating points.
- #pragma GCC target ("avx2") doubles performance of vectorized code.
- #pragma GCC optimize ("trapv") kills the program on integer overflows (excellent for local debugging).

Techniques (A)

techniques.txt	159 lines
Recursion	
Divide and conquer	
Finding interesting points in N log N	
Algorithm analysis	
Master theorem	
Amortized time complexity	
Greedy algorithm	
Scheduling	
Max contiguous subvector sum	
Invariants	
Huffman encoding	
Graph theory	
Dynamic graphs (extra book-keeping)	
Breadth first search	
Depth first search	
* Normal trees / DFS trees	
Dijkstra's algorithm	
MST: Prim's algorithm	
Bellman-Ford	
Konig's theorem and vertex cover	
Min-cost max flow	
Lovasz toggle	
Matrix tree theorem	
Maximal matching, general graphs	
Hopcroft-Karp	
Hall's marriage theorem	
Graphical sequences	
Floyd-Warshall	
Euler cycles	
Flow networks	
* Augmenting paths	
* Edmonds-Karp	
Bipartite matching	
Min. path cover	
Topological sorting	
Strongly connected components	
2-SAT	
Cut vertices, cut-edges and biconnected components	
Edge coloring	
* Trees	
Vertex coloring	
* Bipartite graphs (=> trees)	
* 3^n (special case of set cover)	
Diameter and centroid	
K'th shortest path	
Shortest cycle	
Dynamic programming	
Knapsack	
Coin change	
Longest common subsequence	
Longest increasing subsequence	
Number of paths in a dag	
Shortest path in a dag	
Dynprog over intervals	
Dynprog over subsets	
Dynprog over probabilities	
Dynprog over trees	
3^n set cover	
Divide and conquer	
Knuth optimization	
Convex hull optimizations	
RMQ (sparse table a.k.a 2^k-jumps)	
Bitonic cycle	
Log partitioning (loop over most restricted)	
Combinatorics	

Computation of binomial coefficients	
Pigeon-hole principle	
Inclusion/exclusion	
Catalan number	
Pick's theorem	
Number theory	
Integer parts	
Divisibility	
Euclidean algorithm	
Modular arithmetic	
* Modular multiplication	
* Modular inverses	
* Modular exponentiation by squaring	
Chinese remainder theorem	
Fermat's little theorem	
Euler's theorem	
Phi function	
Frobenius number	
Quadratic reciprocity	
Pollard-Rho	
Miller-Rabin	
Hensel lifting	
Vieta root jumping	
Game theory	
Combinatorial games	
Game trees	
Mini-max	
Nim	
Games on graphs	
Games on graphs with loops	
Grundy numbers	
Bipartite games without repetition	
General games without repetition	
Alpha-beta pruning	
Probability theory	
Optimization	
Binary search	
Ternary search	
Unimodality and convex functions	
Binary search on derivative	
Numerical methods	
Numeric integration	
Newton's method	
Root-finding with binary/ternary search	
Golden section search	
Matrices	
Gaussian elimination	
Exponentiation by squaring	
Sorting	
Radix sort	
Geometry	
Coordinates and vectors	
* Cross product	
* Scalar product	
Convex hull	
Polygon cut	
Closest pair	
Coordinate-compression	
Quadtrees	
KD-trees	
All segment-segment intersection	
Sweeping	
Discretization (convert to events and sweep)	
Angle sweeping	
Line sweeping	
Discrete second derivatives	
Strings	
Longest common substring	
Palindrome subsequences	

Knuth-Morris-Pratt	
Tries	
Rolling polynomial hashes	
Suffix array	
Suffix tree	
Aho-Corasick	
Manacher's algorithm	
Letter position lists	
Combinatorial search	
Meet in the middle	
Brute-force with pruning	
Best-first (A*)	
Bidirectional search	
Iterative deepening DFS / A*	
Data structures	
LCA (2^k-jumps in trees in general)	
Pull/push-technique on trees	
Heavy-light decomposition	
Centroid decomposition	
Lazy propagation	
Self-balancing trees	
Convex hull trick (wcipeg.com/wiki/Convex_hull_trick)	
Monotone queues / monotone stacks / sliding queues	
Sliding queue using 2 stacks	
Persistent segment tree	